



# CS683: Advanced Computer Architecture-2024

L7-Hardware Prefetching and advanced optimizations!!

<https://www.cse.iitb.ac.in/~biswa/courses.html>

<https://www.cse.iitb.ac.in/~biswa/>

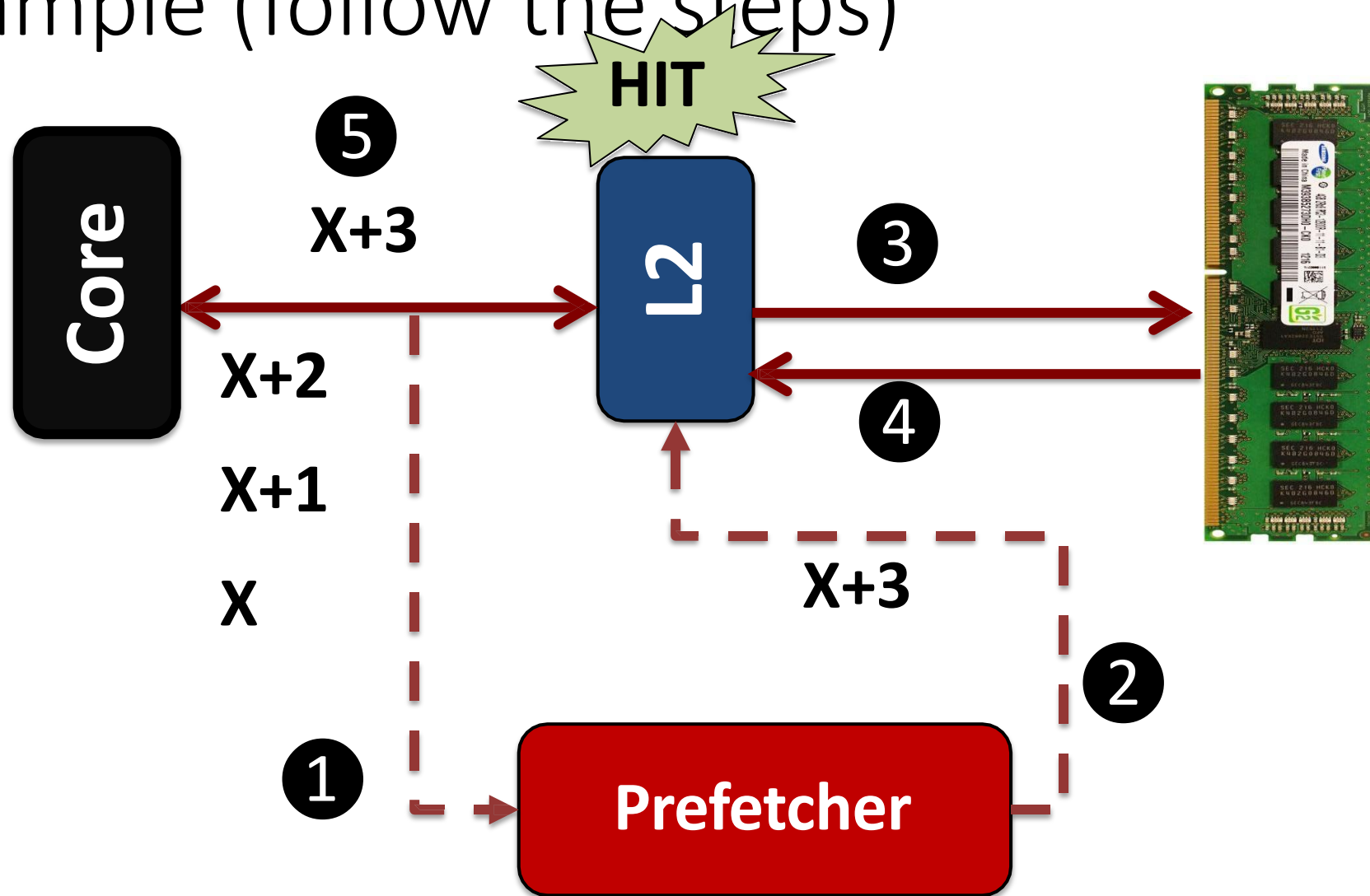


Phones (smart/non-smart)  
on silence plz, Thanks

---

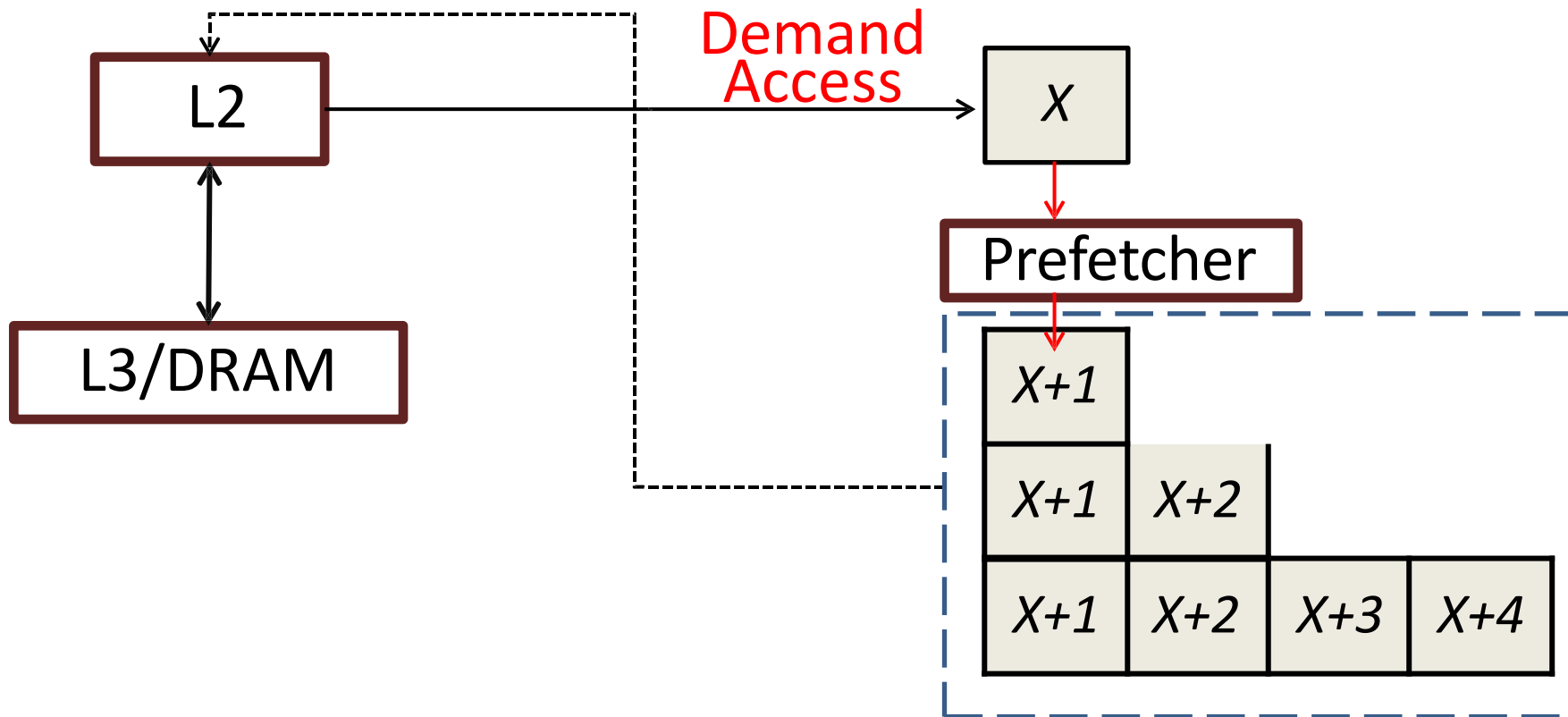
# 1<sup>st</sup> tutorial: Sept 13

An example (follow the steps)



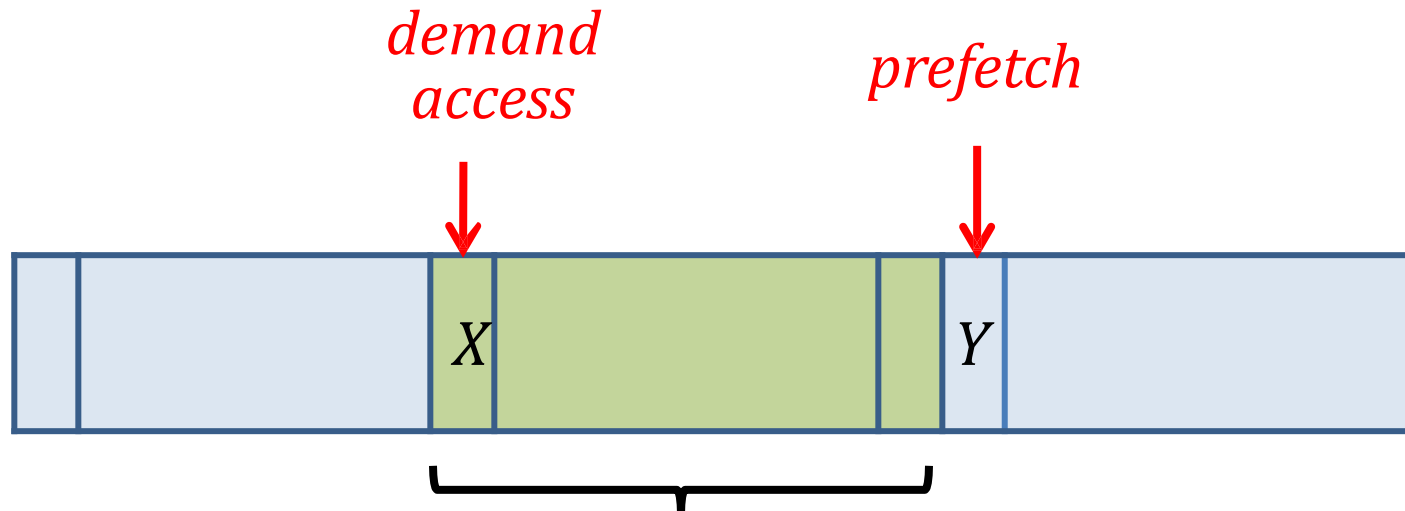
# Prefetch Degree

**Prefetch Degree:** Number of prefetch requests to issue at a given time.  
In this example, it is 1, 2, and 4.



# Prefetch Distance

**Prefetch Distance:** How far ahead of the demand access stream are the prefetch requests issued?



*Prefetch-distance*

$$Y = X + 4$$

$$Y = X + 8$$

$$Y = X + 16$$

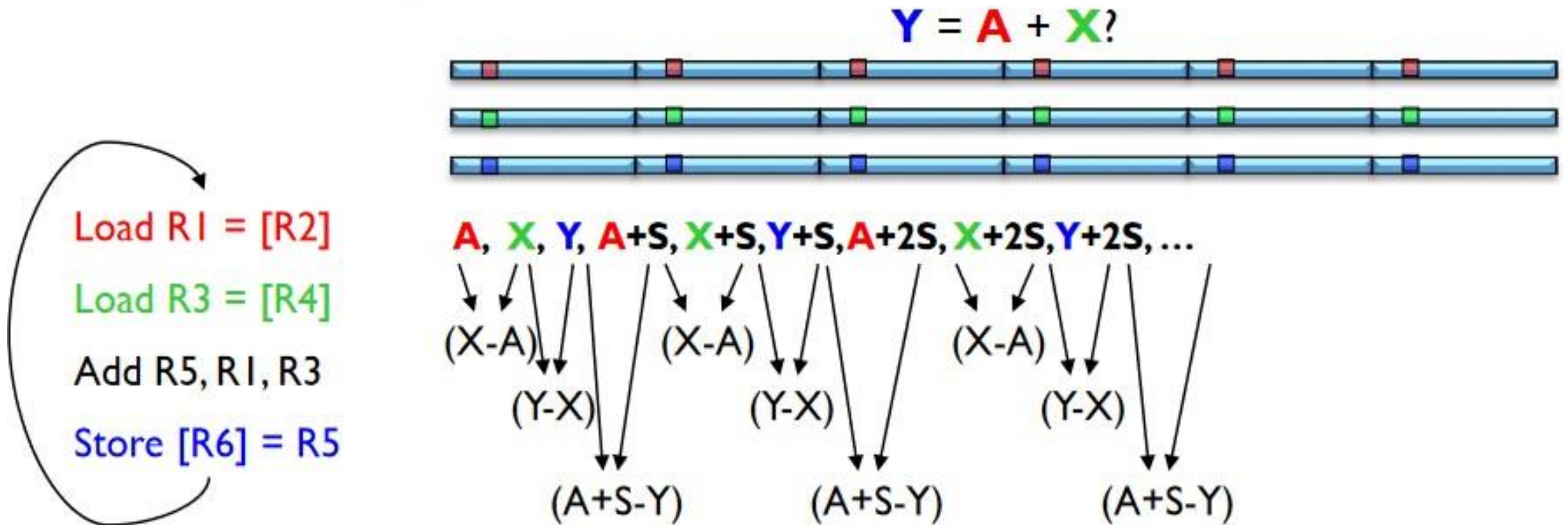
# The Simplest one (Remember it is data prefetching)

**Next Line:** Miss to cache block  $X$ , prefetch  $X+1$ . Degree=1, Distance=1

Works well for L1 Icache and L1 Dcache.

**Next N Line:** Miss to cache block  $X$ , prefetch  $X+1, X+2, \dots, X+N$ ,  
Degree= $N$ , Distance= min. 1 and max.  $N$

What about this? Each load instruction (color) has a stride pattern

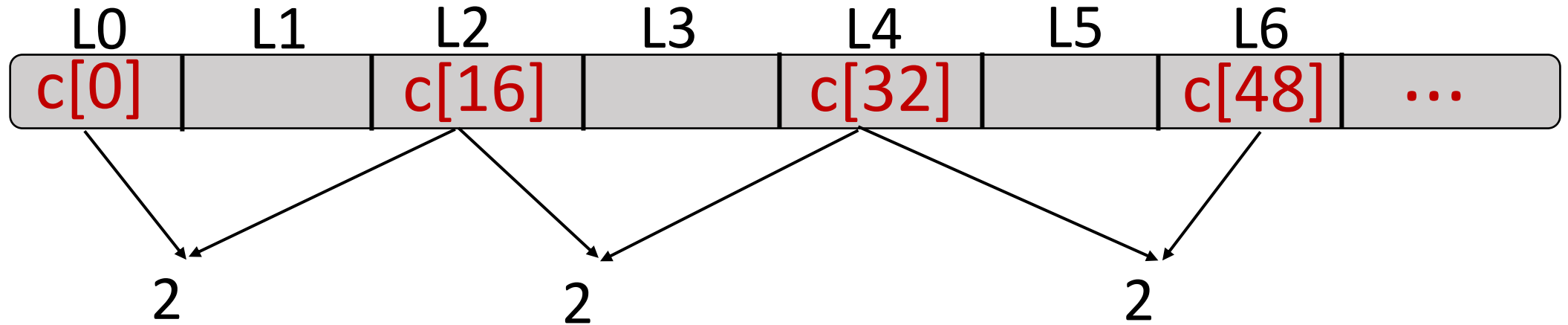


```
for (i = 0; i < N; i=i+16)
```

```
sum += c[i];
```

IP<sub>x</sub>: PC: **LOAD**

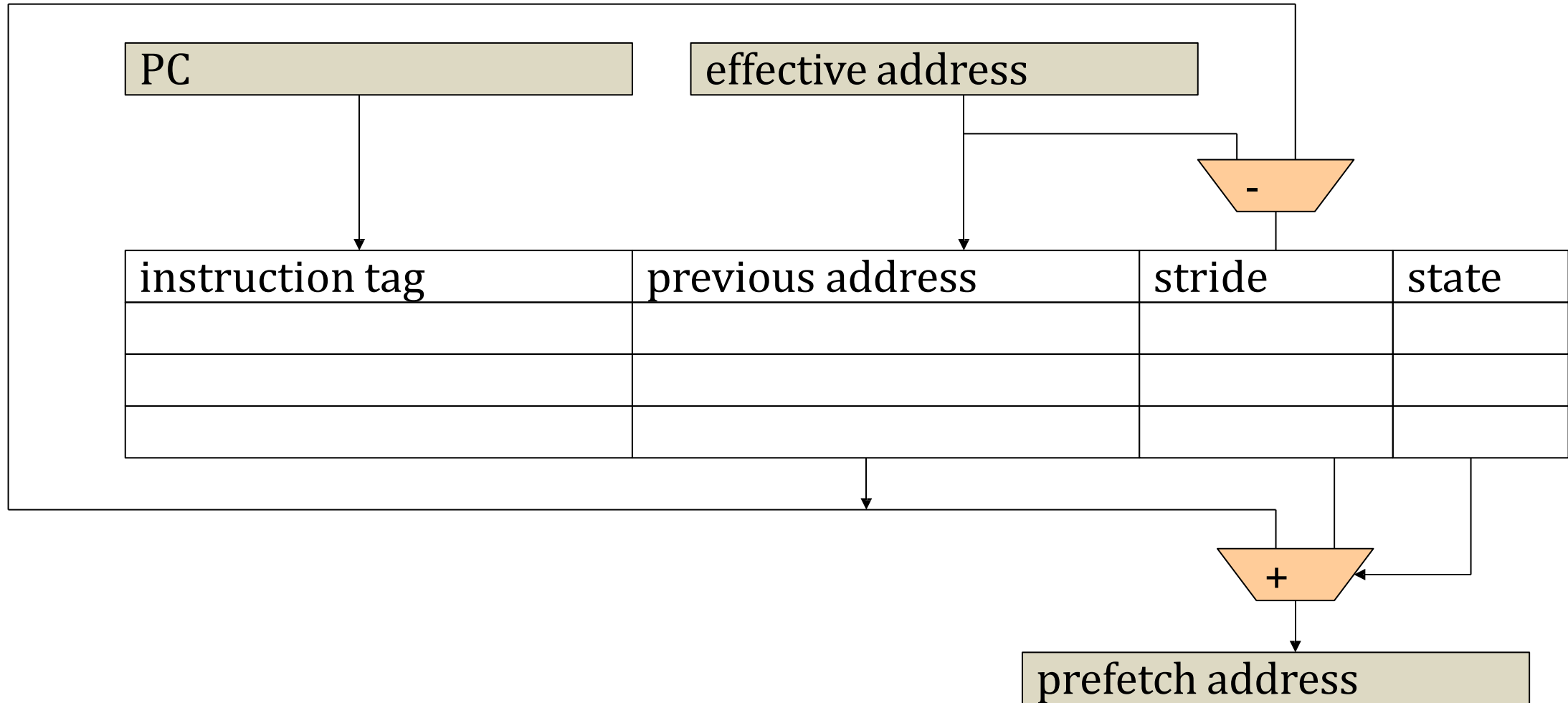
Cache line size of 64B



\*Loops over arrays/structures

Stride=difference in cache line accesses

# IP-stride prefetching [Intel, AMD, ARM]



# IP-stride in Action

```
float a[100][100], b[100][100], c[100][100];  
...  
for ( i = 0; i < 100; i++)  
    for ( j = 0; j < 100; j++)  
        for ( k = 0; k < 100; k++)  
            a[i][j] += b[i][k] * c[k][j];
```

| Tag               | Previous Address | Stride | State   |
|-------------------|------------------|--------|---------|
| <i>ld b[i][k]</i> | 20,000           | 0      | initial |
| <i>ld c[k][j]</i> | 30,000           | 0      | initial |
| <i>ld a[i][j]</i> | 10,000           | 0      | initial |

(b)

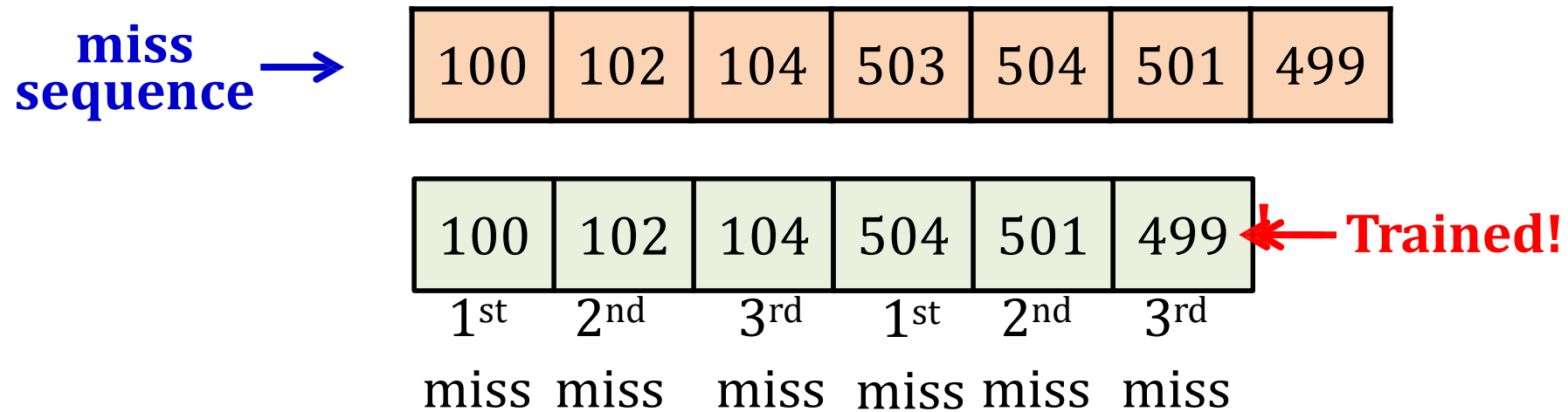
| Tag               | Previous Address | Stride | State     |
|-------------------|------------------|--------|-----------|
| <i>ld b[i][k]</i> | 20,004           | 4      | transient |
| <i>ld c[k][j]</i> | 30,400           | 400    | transient |
| <i>ld a[i][j]</i> | 10,000           | 0      | steady    |

(c)

| Tag               | Previous Address | Stride | State  |
|-------------------|------------------|--------|--------|
| <i>ld b[i][k]</i> | 20,008           | 4      | steady |
| <i>ld c[k][j]</i> | 30,800           | 400    | steady |
| <i>ld a[i][j]</i> | 10,000           | 0      | steady |

(d)

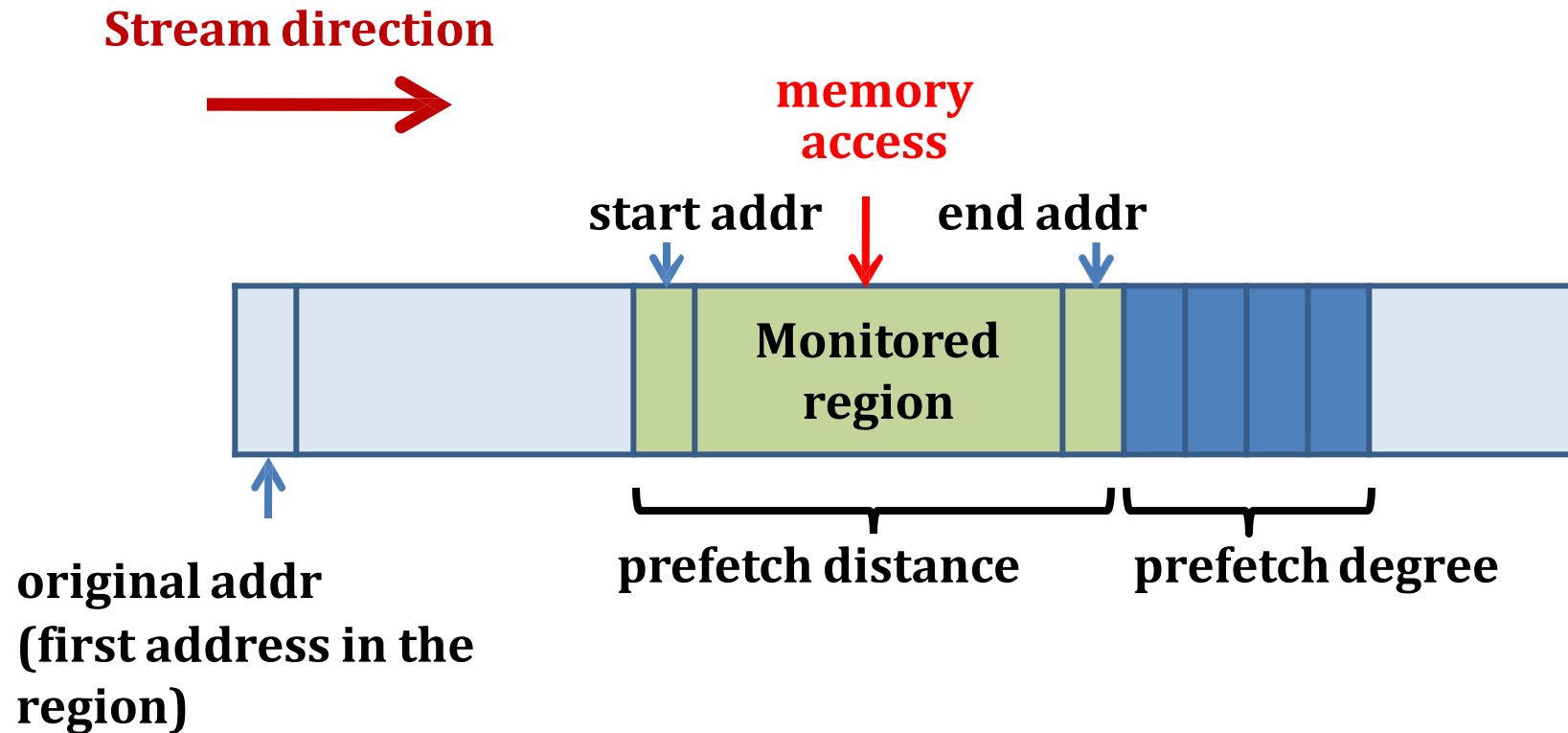
# Stream prefetcher [Intel, IBM, ARM]



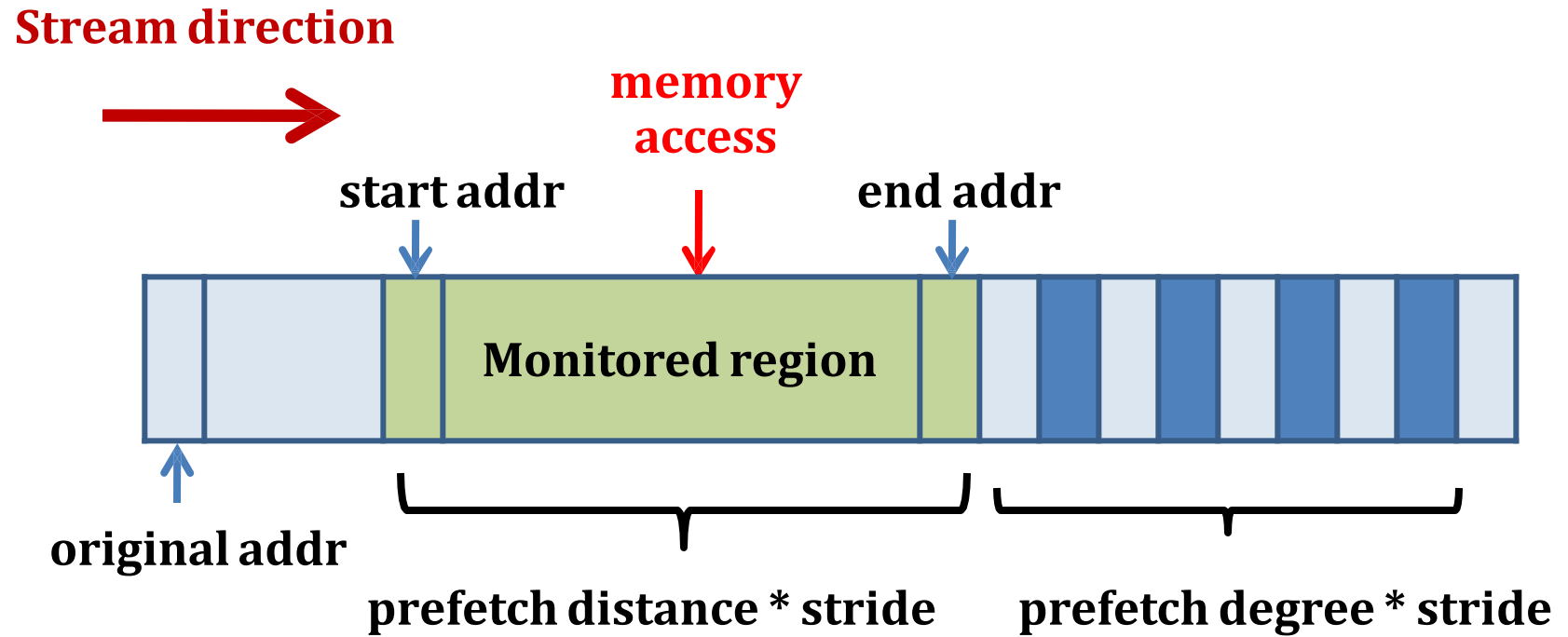
**Training: Consecutive misses in the same direction.**

**Prefetch contiguously in that direction.**

# Stream prefetcher in action



# Stream+Stride



# State-of-the-art prefetchers [ISCA 20, MICRO 22, MICRO 23, MICRO 24] and IITB

## Secure Prefetching for Secure Cache Systems

Sumon Nath\*, Agustín Navarro-Torres<sup>†</sup>, Alberto Ros<sup>‡</sup>, Biswabandan Panda\*

\*Dept. of Computer Science and Engineering, Indian Institute of Technology Bombay, Mumbai, India

<sup>†</sup>Computer Engineering Department, University of Murcia, Murcia, Spain

Email: sumon@cse.iitb.ac.in, agustin.navarro@um.es, aros@ditc.um.es, biswa@cse.iitb.ac.in

## Berti: an Accurate Local-Delta Data Prefetcher

Agustín Navarro-Torres\*, Biswabandan Panda<sup>†</sup>, Jesús Alastruey-Benedé\*,  
Pablo Ibáñez\*, Víctor Viñals-Yúfera\*, and Alberto Ros<sup>‡</sup>

## CLIP: Load Criticality based Data Prefetching for Bandwidth-constrained Many-core Systems

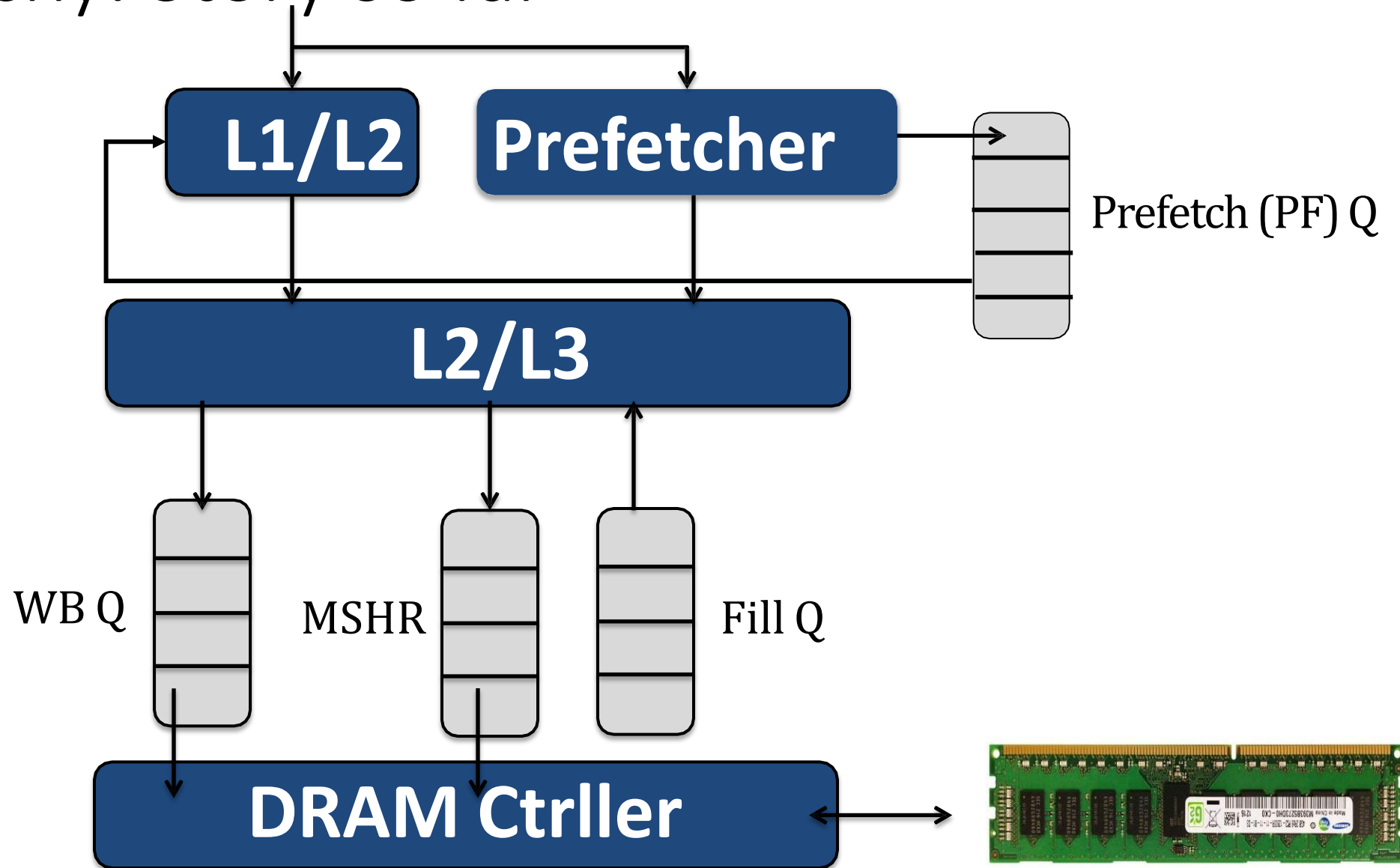
Biswabandan Panda  
biswa@cse.iitb.ac.in  
Indian Institute of Technology Bombay  
Mumbai, India

## Bouquet of Instruction Pointers: Instruction Pointer Classifier-based Spatial Hardware Prefetching

Samuel Pakalapati

Biswabandan Panda

# Cache Hierarchy: Story so far



# Quantifying prefetchers

$$\text{PrefetchAccuracy}(i) = \frac{\text{Prefetch}_{\text{hits}}(i)}{\text{Prefetch}_{\text{issued}}(i)}$$

Prefetched Block in the Cache.

$$\text{Lateness}(i) = \frac{\text{Prefetch}_{\text{late}}(i)}{\text{Prefetch}_{\text{hits}}(i)}$$

Prefetched Block Still on its way

$$\text{Pollution}(i) = \frac{\text{LLC Poll}(i)}{\text{Demand}_{\text{misses}}(i)}$$

Prefetched Block evicted a demand block that will be reused

$$\text{Coverage}(i) = \frac{\text{Prefetch Hits}(i)}{\text{Prefetch Hits}(i) + \text{Demand}_{\text{misses}}(i)}$$

Fraction of misses avoided

# Implementation

- Add a prefetch bit per cache line to distinguish demand blocks from prefetched blocks. Similarly at the MSHR if you want to track anything specific corresponding to prefetched blocks.
- Modern prefetchers play the game between coverage and accuracy, and dynamically choose one metric over other
- Timeliness is extremely critical in multi-core environments (more in next few weeks).

# Summary: Hardware Prefetching

# Some More Advanced optimizations

- Critical word first
- Banked caches
- Joupii's stream buffers
- Victim cache
- Andre's Skewed associative caches
- Pipelined caches

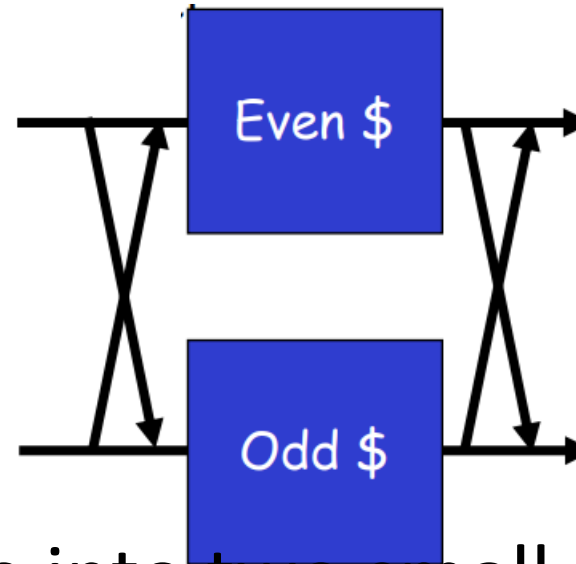
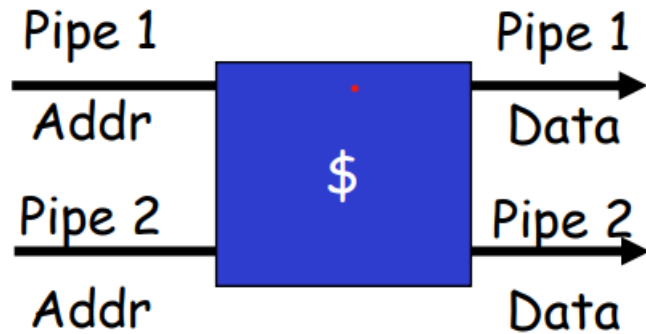
## Critical word first [Text book Chapter 2]

- Respond with the requested data byte (word) first while filling into the cache.
- Makes sense for multi-level cache hierarchy.
- L1 and L2 can get the requested byte before the block is completely filled into L3. Transfer latency is amortized.

# Banked Caches [Chapter 2]

- Split the cache into smaller “banks”
- CPU can send concurrent requests to multiple banks improving overall bandwidth.
- If multiple requests go to same bank then the latency becomes serial.
- Why not multi-ported caches? Ahhh, they are super expensive.

# Journey from multi-port to banked



We can statically divide a cache into two small cache banks. Which bit(s) should be used for dividing the space? Take bits from indexing.

# Pipelined Caches (mostly for L1 caches)

- Comparing index bits, comparing multiple tags, and then respond with the data
- Can we pipeline it? 3-stage pipeline will do
- This will amortize access latency 😊
- Banked cache provides bandwidth 😊